Camlin | Software Team Manager

24 interview questions and model answers • Engineering Manager perspective

Odgovori su na engleskom, komentari i dodatne smernice na srpskom. Prilagodi primere svom stvarnom iskustvu.

Leadership & People Management

How would you describe your leadership style?

My leadership style is based on ownership, transparency, and enabling engineers to succeed. I establish clear expectations, remove blockers, and support professional development. I adapt my approach: junior engineers need more guidance, while seniors benefit from autonomy. I measure my effectiveness through sustainable delivery, engineering quality, and a team that can make good decisions independently.

Proverava se: autonomija tima, coaching i izbegavanje micromanagementa.

How do you handle an underperforming engineer?

I first identify the root cause: unclear expectations, skills, motivation, or organizational obstacles. I discuss specific observations privately, listen to their perspective, agree on measurable goals and checkpoints, and provide support. If improvement does not follow, I use the formal performance process. The approach must be fair to the individual and the team.

Follow-up: šta ako je inženjer odličan tehnički, ali negativno utiče na tim?

How do you keep senior engineers motivated?

I focus on autonomy, meaningful challenges, technical influence, and recognition of impact. I give senior engineers ownership of initiatives, involve them in architecture, and create mentoring opportunities. Regular career conversations help me understand whether they want to pursue architecture, technical leadership, or management.

Proverava se retention i individualni razvoj.

Two engineers on your team are in conflict. How do you resolve it?

I speak with each person individually, distinguish technical disagreement from interpersonal conflict, and then facilitate a discussion around facts and shared goals. For technical disputes, I use objective criteria such as maintainability, risk, and performance. I address unacceptable behavior directly and follow up to confirm that the conflict is resolved.

Proverava se sposobnost za teške razgovore.

How would you approach your first 90 days?

In the first 30 days I learn the people, product, architecture, and delivery process. In days 30-60 I identify recurring bottlenecks and align expectations. In days 60-90 I implement a few measurable improvements and evaluate their effect. I avoid major changes before understanding why existing practices are in place.

Proverava se promišljeno uvođenje promena.

Delivery, Agile & Resource Management

How do you ensure predictable software delivery?

I clarify requirements and acceptance criteria, split work into smaller deliverables, and identify dependencies early. During execution I track risks, blockers, and scope changes, not just activity. When delivery is threatened, I communicate options early: adjust scope, sequence work, remove dependencies, or revise the forecast. Predictability means stakeholders are not surprised.

Proverava se ownership nad isporukom.

How do you handle competing priorities from multiple Product Owners?

I make competing requests visible and compare business value, urgency, dependencies, and capacity. I bring decision-makers together to agree on a single priority order. Engineers should not receive contradictory instructions. If the conflict cannot be resolved at team level, I escalate with clearly stated trade-offs.

Follow-up: šta kada oba Product Ownera tvrde da su njihovi zahtevi kritični?

How do you plan team capacity?

I start from actual availability, accounting for leave, support, maintenance, onboarding, and operational interruptions. I consider skill distribution rather than treating all engineers as interchangeable. Historical throughput helps with forecasts, but individual velocity is not a performance metric. I reserve capacity for reliability and technical debt.

Proverava se realno planiranje resursa.

A critical feature is behind schedule. What do you do?

I identify the root cause and critical path. I evaluate incremental delivery, scope reduction, removal of dependencies, and targeted support. I do not assume adding people will speed up late work. I communicate a realistic revised forecast and later review what should change in planning and execution.

Proverava se donošenje odluka pod pritiskom.

Technical Leadership & Architecture

How would you design a scalable platform for collecting data from thousands of IoT devices?

I would separate device ingestion, durable messaging, stream processing, storage, and analytics. An ingestion gateway authenticates devices and accepts telemetry; a broker buffers events and decouples processing. Consumers validate, enrich, and persist data. I would consider time-series storage, retention, backpressure, retries, dead-letter queues, and observability. Real-time alerting and historical analytics may need different pipelines.

Follow-up: broker outage, duplicate/out-of-order events, offline devices, idempotency, sequence numbers, consumer lag.

How do you decide between microservices and a modular monolith?

I evaluate domain boundaries, team ownership, independent deployment and scaling needs, and operational maturity. A modular monolith can preserve boundaries with less distributed-systems complexity. Microservices are justified when independent scaling, release cadence, or isolation provides clear value. I avoid distribution for its own sake.

Proverava se razumevanje trade-off odluka.

How would you improve the reliability of an existing SaaS platform?

I establish a baseline from incidents, availability, latency, and error rates. I identify the most costly failure modes and prioritize changes such as better telemetry, resilient communication, rollback, automated checks, and recovery procedures. I define useful SLOs and verify that changes improve outcomes.

SLI = indikator; SLO = ciljni nivo; SLA = ugovorna obaveza.

How do you introduce engineering standards across multiple teams?

I start with inconsistency that causes real problems, such as security gaps, fragile deployments, or missing observability. I co-design a small set of standards with senior engineers, provide templates and automation, and permit justified exceptions. The objective is useful consistency, not uniformity for its own sake.

Proverava se saradnja i pragmatična standardizacija.

What metrics would you use to evaluate engineering performance?

I look at delivery lead time and predictability, escaped defects and rework, operational reliability, deployment frequency, change failure rate, and recovery time. I also assess team health, retention, and skill development. I avoid using lines of code or commit counts as individual productivity scores.

Proverava se da li metričke podatke koristiš kao dijagnostiku, ne kao oružje.

Hiring, Retention & Career Development

How do you interview and hire software engineers?

I define role outcomes and use structured interviews with consistent criteria. For seniors I test reasoning, production experience, trade-offs, communication, and ownership. I compare observed evidence against requirements rather than relying on impressions. I also assess collaboration and learning ability.

Follow-up: odličan tehničar sa slabom komunikacijom?

One of your best engineers wants to leave. What do you do?

I seek to understand whether the issue is growth, recognition, workload, compensation, leadership, or the work itself. I discuss realistic options without making promises I cannot keep. Retention is continuous: meaningful ownership, recognition, career conversations, and a healthy environment must exist before someone announces they are leaving.

Proverava se dugoročna retention strategija.

How do you develop junior and mid-level engineers into senior engineers?

I gradually increase ownership through design discussions, reviews, incident investigation, cross-team collaboration, and independent decision-making. I provide feedback and mentoring while reducing unnecessary supervision. Seniority means navigating ambiguity, anticipating risk, choosing trade-offs, and enabling others, not just writing complex code.

Proverava se razvoj ljudi kroz odgovornost.

Situational Questions

Your team consistently misses deadlines and Product blames Engineering. How do you respond?

I avoid blame and review evidence: estimates, scope changes, dependencies, capacity, and technical debt. I take responsibility for unrealistic engineering commitments and make the impact of changing requirements transparent. I align Product and Engineering on a shared improvement plan and clear forecasting.

Proverava se saradnja pod pritiskom.

Production is down, engineers are unavailable, and a customer is escalating. What do you do?

I establish incident command, assign investigation, mitigation, and communication owners, and prioritize safe service restoration. Rollback or isolation may be safer than a rushed fix. I give regular factual updates, then conduct a blameless review with corrective actions and owners.

Follow-up: šta ako je incident izazvao najiskusniji inženjer?

Management asks you to increase productivity by 30%. What do you do?

I clarify what productivity means and establish a credible baseline. I examine wait time, rework, context switching, deployment friction, and unclear requirements. I target systemic waste rather than longer hours, run measurable improvements, and report both gains and trade-offs.

Proverava se da li razumeš održivu produktivnost.

Questions About Your Background

Tell me about yourself and why are you interested in this position?

I work as a Software Engineer Specialist at FIS, with a background in enterprise .NET systems and delivery ownership. My responsibilities include coordinating engineering work, mentoring, removing blockers, and resolving client-impacting issues. I enjoy combining technical understanding with helping teams deliver reliably. This role interests me because it combines people development, delivery accountability, and technically meaningful software in the electrical-grid domain.

Prilagodi konkretnim primerima koje možeš da dokažeš.

What is the difference between a Tech Lead and an Engineering Manager?

A Tech Lead primarily owns technical direction and engineering execution. An Engineering Manager is responsible for the people and organizational environment: hiring, performance, development, capacity, team health, and delivery effectiveness. Technical credibility matters, but the manager should enable technical decision-makers rather than approve every design.

Proverava se razumevanje razlike između dve karijerne putanje.

Why do you want to move from engineering into management?

I enjoy technical work, but I have found that some of my highest-impact contributions come from enabling engineers, removing ambiguity, improving coordination, and taking responsibility for outcomes. Management is a different discipline, not simply a promotion away from coding. I want to develop that discipline while using my engineering experience to support better decisions.

Izbegni odgovor da želiš samo sledeći nivo titule.

How much formal people management experience do you have?

My background is primarily in software engineering, while my responsibilities have expanded into team coordination, mentoring, delivery planning, and removing blockers. I can describe the leadership activities I have directly owned. I also understand that formal line management includes hiring decisions, performance reviews, and structured career development, and I would distinguish my direct experience from areas where I am still developing.

Važno: ne predstavljaj neformalno vođenje kao formalni line management.

Priprema: 5 STAR primera

Pripremi konkretne priče o konfliktu u timu, delivery problemu, razvoju inženjera, produkcionom incidentu i kompromisu između brzine i kvaliteta. Za svaku definiši Situation, Task, Action i Result.